

Numerical equations

Speakers: Dzagaryan A. S.

group FS2-61B

3 июня 2025 г.



dep. Applied mathematics FS-2
Group FS2-61B

- 1) Why do we need a numerical equation?
- 2) What is derivative of the function
- 3) What is integral of the function
- 4) What is linear system.
- 5) Example of numerical equation.

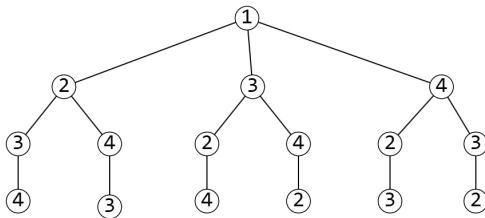


Рис. Дерево строится по городам

Границы

Верхней границей $\bar{t}$ называется наименьшая стоимость путей, которые были рассмотрены до текущего момента времени.
Нижней границей $\underline{t}$ узла называется число, которое оценивает стоимость пути снизу.

Условие отсечения ветвей

Если $\underline{t} \geq \bar{t}$, отсекаем все поддереву данного узла.

Проблема

$\underline{t}$ и $\bar{t}$ имеют разные размерности, потому что верхняя граница оценивает длину всего пути, а нижняя оценивает длину его части.

Решение

Нужно строить дерево, так чтобы нижняя граница оценивала стоимость всего пути.

Построение дерева

Дерево строится по ребрам, в отличие от наивного алгоритма, где дерево строилось по городам.

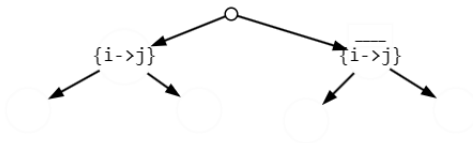


Рис. Дерево строится по ребрам

Отсечение

Надо отсекаать правые ветви, т.к. в правом поддереве размерность постоянна, а в левом поддереве размерность уменьшается на 1.

Редуцирование строк

Редуцирование строк – преобразование элементов матрицы фиксированной строки i по следующей формуле

$$Matrix[i][j] = Matrix[i][j] - \min_{k=1 \dots n} Matrix[i][k] \text{ для } \forall j = 1 \dots n.$$

Редуцирование столбцов

Редуцирование столбцов – преобразование элементов матрицы фиксированного столбца j по следующей формуле

$$Matrix[i][j] = Matrix[i][j] - \min_{k=1 \dots n} Matrix[k][j] \text{ для } \forall i = 1 \dots n.$$

Утверждение

После редуцирования всех строк и столбцов матрицы

1) в каждой строке и столбце будет хотя бы один нулевой элемент.

2) оптимальность пути не изменится

3) Нижняя граница увеличится на величину

$$\sum_{i=1\dots n} \min_{k=1\dots n} (Matrix[i][k]) + \sum_{j=1\dots n} \min_{k=1\dots n} (Matrix[k][j])$$

Выбор ребра

Среди ребер $i \rightarrow j$ с нулевым весом нужно выбрать, то для которого выражение

$$\min_{k=1\dots n, k \neq j} (Matrix[i][k]) + \min_{k=1\dots n, k \neq i} (Matrix[k][j]) \rightarrow \max$$

BranchAndBound

- 1: Редуцируем строки и столбцы матрицы
- 2: Увеличиваем нижнюю границу
- 3: **if** $\underline{t} \geq \bar{t}$ **then**
- 4: Выход
- 5: **end if**
- 6: **if** путь полный **then**
- 7: Обновить $\bar{t}$
- 8: Обновить Лучший путь
- 9: Выход
- 10: **end if**
- 11: Поиск ребра с наибольшим штрафом
- 12: BranchAndBound(не берем найденное ребро)
- 13: BranchAndBound(берем найденное ребро)

Окрестность Swap-city

Swap-city

Выбираются два города i_k и i_l и они меняются местами. Таким образом окрестность решения s имеет вид $N(s) = \{s' \in S \mid \forall k, l: s' = \{i_1, \dots, i_{k-1}, i_l, i_{k+1}, \dots, i_{l-1}, i_k, i_{l+1}, \dots, i_n\}\}$.

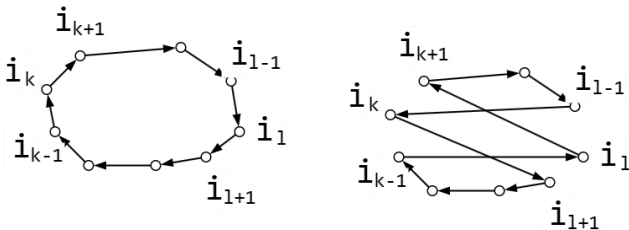


Рис. swapcity

Мощность такой окрестности $|N(s)| = \frac{n(n-1)}{2}$

Оптимальная окрестность

Оптимальной окрестностью решения s имеет вид

$$N^*(s) = \{s' \in N(s) \mid f(s') < f(s)\}$$

Общая схема алгоритма локального спуска

Шаг 1. Выбрать начальное решение $s \in S$, вычислить $f(s)$.

Шаг 2. Выбрать решение s' из окрестности $N^*(s)$.

Шаг 3. Если решение s' найдено, то $s := s'$ и вернуться на шаг 2
иначе Stop, s – локальный минимум

Проблема

Проблема алгоритма локального поиска заключается в том, что он может застрять в локальном оптимуме и не найти глобальный оптимум

Решение проблемы застревания в локальном оптимуме

Воспользоваться tabu list. Он будет хранить информацию о пройденных этапах поиска и позволит алгоритму избегать повторных посещений уже исследованных областей пространства решений.

Общая схема алгоритма поиска с запретами

Шаг 1. Выбрать начальное решение $s \in S$, вычислить $f(s)$

Шаг 2. Пока не выполнен критерий остановки выполнить:

выбрать наилучшее решение $s' \in N(s) \setminus \text{Tabulist}$

$s := s'$

по решению s' обновить Tabulist

Шаг 3. Выдать в качестве ответа лучшее из найденных решений.

Формулировка подзадачи

Пусть знаем длину какого-то пути s . Нужно посчитать длину пути нового пути s' после перестановки городов с индексами i_k и i_l в пути s

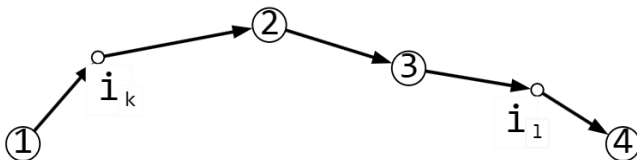


Рис. swapcityexample

Рассмотрим случай когда индексы городов, которые нужно обменять местами не соседние т.е. $|i_k - i_l| > 1$

$$\begin{aligned} Cost_{new} = & cost(\{p[1], p[i_k]\}, \{p[i_k], p[2]\}, \{p[3], p[i_l]\}, \{p[i_l], p[4]\}) \\ & + cost(\{p[1], p[i_l]\}, \{p[i_l], p[2]\}, \{p[3], p[i_k]\}, \{p[i_k], p[4]\}) \end{aligned}$$

Рассмотрим случай когда города, которые нужно обменять соседние.

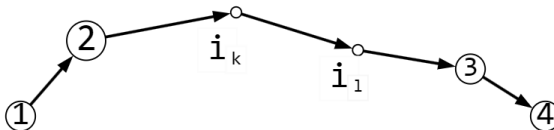


Рис. swapexample1

$$\begin{aligned} Cost_{new} = Cost &- \text{СТОИМОСТЬ}(\{p[2], p[i_k]\}, \{p[i_k], p[i_1]\}, \{p[i_1], p[3]\}) \\ &+ \text{СТОИМОСТЬ}(\{p[2], p[i_1]\}, \{p[i_1], p[i_k]\}, \{p[i_k], p[3]\}) \end{aligned}$$

